



## C++ 八级

2025 年 09 月

### 1 单选题（每题 2 分，共 30 分）

**第 1 题** 小杨想点一杯奶茶外卖，但还差5元起送。于是，小杨决定点一些小料。可选的小料包括：珍珠1元、椰果2元、奶冻3元、奶盖4元。每种小料最多点1份。请问共有多少种满足起送条件的点小料方案？（ ）。

- ☐ A. 16
- ☐ B. 10
- ☐ C. 9
- ☐ D. 7

**第 2 题** 小杨和小刘是好朋友，她们在逛商场时发现新设置的大头贴自拍机，于是决定一起拍一组照片。一组照片包括4张，这4张照片没有顺序区分。拍每张照片时，可以选择有相框或无相框、两人可以分别选择有头饰或无头饰、还可以从2种位置（小杨在左，或小刘在左）中选出一种。她们不希望一组照片中出现完全相同的相框、头饰、位置的组合。请问一组照片共有多少种不同的方案？（ ）。

- ☐ A. 1820
- ☐ B. 70
- ☐ C. 24
- ☐ D. 16

**第 3 题** 下列关于C++类的说法，错误的是（ ）。

- ☐ A. 派生类对象占用的内存总是不小于基类对象。
- ☐ B. 派生类可以不实现基类的虚函数。
- ☐ C. 如果一个类包含纯虚函数，则它不能包含成员变量。
- ☐ D. 如果一个类包含纯虚函数，则不能用它定义对象。

**第 4 题** 下列关于树和图的说法，错误的是（ ）。

- ☐ A. 每个连通图都存在生成树。
- ☐ B. 每个存在生成树的有向图，都一定是强连通的。
- ☐ C. 保留树的所有节点，并把树的每个节点指向其父节点，则可以将树转换为一个有向弱连通图。
- ☐ D. 保留树的所有节点，并把树的每个节点指向其子节点，则可以将树转换为一个有向无环图。

**第 5 题** 一对夫妻生男生女的概率相同。这对夫妻希望儿女双全。请问这对夫妻生下三个孩子时，实现儿女双全的概率是多少？（ ）。

- ☐ A.  $\frac{1}{4}$
- ☐ B.  $\frac{1}{2}$
- ☐ C.  $\frac{3}{4}$
- ☐ D.  $\frac{7}{8}$

第6题 二项式 $(x+y)^6$ 的展开式中 $x^2y^4$ 项的系数是（ ）。

- ☐ A. 720
- ☐ B. 120
- ☐ C. 20
- ☐ D. 15

第7题 对一个包含 $V$ 个顶点、 $E$ 条边的图，执行广度优先搜索，其最优时间复杂度是（ ）。

- ☐ A.  $O(V)$
- ☐ B.  $O(V+E)$
- ☐ C.  $O(V^2)$
- ☐ D.  $O(E)$

第8题 以下关于贪心法和动态规划的说法中，错误的是（ ）。

- ☐ A. 动态规划能解决大部分多阶段决策问题。
- ☐ B. 对特定的问题，贪心法不一定适用。
- ☐ C. 当特定的问题适用贪心法时，通常比动态规划的时间复杂度更低。
- ☐ D. 对很多问题，递推实现和递归实现动态规划方法的时间复杂度相当。

第9题 下面程序的输出为（ ）。

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      int N = 15, cnt = 0;
5      for (int x = 1; x + x + x <= N; x++)
6          for (int y = x; x + y + y <= N; y++)
7              for (int z = y; x + y + z <= N; z++)
8                  cnt++;
9      cout << cnt << endl;
10     return 0;
11 }
```

- ☐ A. 45
- ☐ B. 102
- ☐ C. 174
- ☐ D. 3375

第10题 下面程序的时间复杂度为（ ）。

```

1  int primes[MAXP], num = 0;
2  bool isPrime[MAXN] = {false};
3  void sieve() {
4      for (int n = 2; n <= MAXN; n++) {
5          if (!isPrime[n])
6              primes[num++] = n;
7          for (int i = 0; i < num && n * primes[i] <= MAXN; i++) {
8              isPrime[n * primes[i]] = true;
9              if (n % primes[i] == 0)
10                 break;
11          }
12      }
13  }

```

- ☐ A.  $O(n \log n)$
- ☐ B.  $O(n \log \log n)$
- ☐ C.  $O(n)$
- ☐ D.  $O(\log n)$

第 11 题 下列Dijkstra算法，假设图  $graph$  中顶点数  $v$ 、边数  $e$ ，则程序的时间复杂度为（ ）。

```

1  typedef struct Edge {
2      int in, out;    // 从下标in顶点到下标out顶点的边
3      int len;        // 边长度
4      struct Edge * next;
5  } Edge;
6  // v: 顶点个数, graph: 出边邻接表, start: 起点下标, dis: 输出每个顶点的最短距离
7  void dijkstra(int v, Edge * graph[], int start, int * dis) {
8      const int MAX_DIS = 0x7fffffff;
9      for (int i = 0; i < v; i++)
10         dis[i] = MAX_DIS;
11     dis[start] = 0;
12     int * visited = new int[v];
13     for (int i = 0; i < v; i++)
14         visited[i] = 0;
15     visited[start] = 1;
16     for (int t = 0; ; t++) {
17         int min = MAX_DIS, minv = -1;
18         for (int i = 0; i < v; i++) {
19             if (visited[i] == 0 && min > dis[i]) {
20                 min = dis[i];
21                 minv = i;
22             }
23         }
24         if (minv < 0)
25             break;
26         visited[minv] = 1;
27         for (Edge * e = graph[minv]; e != NULL; e = e->next)
28             if (dis[e->out] > e->len)
29                 dis[e->out] = e->len;
30     }
31     delete[] visited;
32 }

```

- ☐ A.  $O(v^2)$
- ☐ B.  $O(v \log v + e)$
- ☐ C.  $O((v + e) \log v)$
- ☐ D.  $O(v + e)$

第 12 题 下面 count\_triple 函数的时间复杂度为( )。

```

1  int gcd(int m, int n) {
2      if (m == 0) return n;
3      return gcd(n % m, m);
4  }
5  int count_triple(int n) {
6      int cnt = 0;
7      for (int v = 1; v * v * 4 <= n; v++)
8          for (int u = v + 1; u * (u + v) * 2 <= n; u += 2)
9              if (gcd(u, v) == 1) {
10                 int a = u * u - v * v;
11                 int b = u * v * 2;
12                 int c = u * u + v * v;
13                 cnt += n / (a + b + c);
14             }
15      return cnt;
16  }

```

- ☐ A.  $O(n^2)$
- ☐ B.  $O(n^2 \log n)$
- ☐ C.  $O(n)$
- ☐ D.  $O(n \log n)$

第 13 题 下面 merge\_sort 函数试图实现归并排序算法，横线处应该填入的是（ ）。

```

1  #include <vector>
2  using namespace std;
3  void merge_sort(vector<int> & arr, int left, int right) {
4      if (right - left <= 1)
5          return;
6
7      int mid = (left + right) / 2;
8      merge_sort(_____); // 在此处填入选项
9      merge_sort(_____); // 在此处填入选项
10
11     vector<int> temp(right - left);
12     int i = left, j = mid, k = 0;
13     while (i < mid && j < right)
14         if (arr[i] <= arr[j])
15             temp[k++] = arr[i++];
16         else
17             temp[k++] = arr[j++];
18     while (i < mid)
19         temp[k++] = arr[i++];
20     while (j < right)
21         temp[k++] = arr[j++];
22     for (i = left, k = 0; i < right; ++i, ++k)
23         arr[i] = temp[k];
24 }

```

☐ A.

```

1  arr, left, mid
2  arr, mid, right

```

☐ B.

```

1  arr, left, mid + 1
2  arr, mid + 1, right

```

☐ C.

```
1 arr, left, mid
2 arr, mid + 1, right
```

☐ D.

```
1 arr, left, mid + 1
2 arr, mid + 1, right + 1
```

第 14 题 下面Prim算法程序中，横线处应该填入的是（ ）。

```
1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4 using namespace std;
5 int prim(vector<vector<int>> & graph, int n) {
6     vector<int> key(n, INT_MAX);
7     vector<int> parent(n, -1);
8     key[0] = 0;
9     for (int i = 0; i < n; i++) {
10         int u = min_element(key.begin(), key.end()) - key.begin();
11         if (key[u] == INT_MAX)
12             break;
13         for (int v = 0; v < n; v++) {
14             if (_____) { // 在此处填入选项
15                 key[v] = graph[u][v];
16                 parent[v] = u;
17             }
18         }
19     }
20     int sum = 0;
21     for (int i = 0; i < n; i++) {
22         if (parent[i] != -1) {
23             cout << "Edge: " << parent[i] << " - " << i << " Weight: " << key[i] << endl;
24             sum += key[i];
25         }
26     }
27     return sum;
28 }
29 int main() {
30     int n, m;
31     cin >> n >> m;
32     vector<vector<int>> graph(n, vector<int>(n, 0));
33     for (int i = 0; i < m; i++) {
34         int u, v, w;
35         cin >> u >> v >> w;
36         graph[u][v] = w;
37         graph[v][u] = w;
38     }
39     int result = prim(graph, n);
40     cout << "Total weight of the minimum spanning tree: " << result << endl;
41     return 0;
42 }
```

☐ A.

```
1 graph[u][v] >= 0 && key[v] > graph[u][v]
```

☐ B.

```
1 graph[u][v] <= 0 && key[v] > graph[u][v]
```

☐ C.

```
1 graph[u][v] == 0 && key[v] > graph[u][v]
```

☐ D.

```
1 | graph[u][v] != 0 && key[v] > graph[u][v]
```

第 15 题 下面的程序使用出边邻接表表达的带权无向图，则从顶点0到顶点3的最短距离为（ ）。

```
1 | #include <vector>
2 | using namespace std;
3 | class Edge {
4 | public:
5 |     int dest;
6 |     int weight;
7 |     Edge(int d, int w) : dest(d), weight(w) {}
8 | };
9 | class Graph {
10 | private:
11 |     int num_vertex;
12 |     vector<vector<Edge>> vve;
13 | public:
14 |     Graph(int v) : num_vertex(v), vve(v) {}
15 |     void addEdge(int s, int d, int w) {
16 |         vve[s].emplace_back(d, w);
17 |         vve[d].emplace_back(s, w)
18 |     }
19 | };
20 | int main() {
21 |     Graph g(4);
22 |     g.addEdge(0, 1, 8);
23 |     g.addEdge(0, 2, 5);
24 |     g.addEdge(1, 2, 1);
25 |     g.addEdge(1, 3, 3);
26 |     g.addEdge(2, 3, 7);
27 |     return 0;
28 | }
```

☐ A. 12

☐ B. 11

☐ C. 10

☐ D. 9

## 2 判断题（每题 2 分，共 20 分）

| 题号 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|----|---|---|---|---|---|---|---|---|---|----|
| 答案 | × | × | × | √ | × | √ | √ | √ | × | ×  |

第 1 题 C++语言中，表达式 '9' ^ 3 的结果值为 '999' 。

第 2 题 下列C++语言代码，能够安全地输出 arr[5] 的值。

```
1 | int n = 5;
2 | int arr[n] = {1, 2, 3};
3 | std::cout << arr[5];
```

第 3 题 对n个元素的数组进行排序，最差情况的时间复杂度为  $O(n^2)$ 。

第 4 题 有4个红球、3个蓝球和2个绿球排成一排（相同色球视为完全相同），则不同的排列方案数为1260种。

第 5 题 使用 math.h 或 cmath 头文件中的函数，对于 int 类型的变量 x，表达式 fabs(x) 和 sqrt(x \* x) 的结果总是近似相等的。

第6题 运算符重载是C++语言静态多态的一种典型体现，而使用C语言则无法实现运算符重载。

第7题 存在一个简单无向图满足：顶点数为6，边数为8，6个顶点的度数分别为3、3、3、3、2、2。

第8题 已知两个 `double` 类型的变量 `r` 和 `theta` 分别表示一个扇形的圆半径及圆心角（弧度），则扇形的周长可以通过表达式  $(2 + \text{theta}) * r$  求得。

第9题 Dijkstra算法的时间复杂度为  $O(V^2)$ ，其中  $V$  为图中顶点的数量。

第10题 从32名学生中选出2人分别担任男生班长和女生班长（男生班长必须是男生，女生班长必须是女生），则共有  $C(32, 2)/2$  种不同的选法。

### 3 编程题（每题 25 分，共 50 分）

#### 3.1 编程题 1

- 试题名称：最短距离
- 时间限制：1.0 s
- 内存限制：512.0 MB

##### 3.1.1 题目描述

给定正整数  $p, q$  以及常数  $N = 10^{18}$ 。现在构建一张包含  $N$  个结点的带权无向图，结点依次以  $1, 2, \dots, N$  编号。对于任意满足  $1 \leq u < v \leq N$  的  $u, v$ ，向图中加入一条连接结点  $u$  与结点  $v$  的无向边，边权取决于  $u, v$  是否互质：

- 若  $u, v$  互质（即  $u, v$  的最大公因数为 1），则连接结点  $u$  与结点  $v$  的无向边长度为  $p$ ；
- 否则连接结点  $u$  与结点  $v$  的无向边长度为  $q$ 。

现在给定  $n$  组询问，第  $i$  ( $1 \leq i \leq n$ ) 组询问给定两个正整数  $a_i, b_i$ ，你需要回答结点  $a_i$  与结点  $b_i$  之间的最短距离。

##### 3.1.2 输入格式

第一行，三个正整数  $n, p, q$ ，分别表示询问数量，结点编号互质时的边权，以及结点编号不互质时的边权。

接下来  $n$  行，每行两个正整数  $a_i, b_i$ ，表示一组询问。

##### 3.1.3 输出格式

输出共  $n$  行，每行一个整数，表示结点  $a_i$  与结点  $b_i$  之间的最短距离。

##### 3.1.4 样例

###### 3.1.4.1 输入样例 1

```
1 | 4 4 3
2 | 1 2
3 | 2 3
4 | 4 2
5 | 3 5
```

###### 3.1.4.2 输出样例 1

```
1 | 4
2 | 4
3 | 3
4 | 4
```

### 3.1.4.3 输入样例 2

|   |       |
|---|-------|
| 1 | 5 2 6 |
| 2 | 1 2   |
| 3 | 2 3   |
| 4 | 4 2   |
| 5 | 3 5   |
| 6 | 6 6   |

### 3.1.4.4 输出样例 2

|   |   |
|---|---|
| 1 | 2 |
| 2 | 2 |
| 3 | 4 |
| 4 | 2 |
| 5 | 0 |

### 3.1.5 数据范围

对于 30% 的测试点，保证  $1 \leq n \leq 10$ ， $1 \leq a_i, b_i \leq 50$ 。

对于另外 30% 的测试点，保证  $1 \leq a_i, b_i \leq 250$ 。

对于所有测试点，保证  $1 \leq n \leq 10^4$ ， $1 \leq a_i, b_i \leq 10^9$ ， $1 \leq p, q \leq 10^9$ 。

### 3.1.6 参考程序



## 3.2 编程题 2

- 试题名称：最小生成树
- 时间限制：1.0 s
- 内存限制：512.0 MB

### 3.2.1 题目描述

给定一张包含  $n$  个结点  $m$  条边的带权连通无向图，结点依次以  $1, 2, \dots, n$  编号，第  $i$  条边 ( $1 \leq i \leq m$ ) 连接结点  $u_i$  与结点  $v_i$ ，边权为  $w_i$ 。

对于每条边，请你求出从图中移除该条边后，图的最小生成树中所有边的边权和。特别地，若移除某条边后图的最小生成树不存在，则输出  $-1$ 。

### 3.2.2 输入格式

第一行，两个正整数  $n, m$ ，分别表示图的结点数与边数。

接下来  $m$  行中的第  $i$  行 ( $1 \leq i \leq m$ ) 包含三个正整数  $u_i, v_i, w_i$ ，表示图中连接结点  $u_i$  与结点  $v_i$  的边，边权为  $w_i$ 。

### 3.2.3 输出格式

输出共  $m$  行，第  $i$  行 ( $1 \leq i \leq m$ ) 包含一个整数，表示移除第  $i$  条边后，图的最小生成树中所有边的边权和。若移除第  $i$  条边后图的最小生成树不存在，则输出  $-1$ 。

### 3.2.4 样例

#### 3.2.4.1 输入样例 1

|   |       |
|---|-------|
| 1 | 5 5   |
| 2 | 1 2 4 |
| 3 | 2 3 3 |
| 4 | 3 4 1 |
| 5 | 2 5 2 |
| 6 | 3 1 8 |

#### 3.2.4.2 输出样例 1

|   |    |
|---|----|
| 1 | 14 |
| 2 | 15 |
| 3 | -1 |
| 4 | -1 |
| 5 | 10 |

#### 3.2.4.3 输入样例 2

|    |       |
|----|-------|
| 1  | 6 10  |
| 2  | 1 2 6 |
| 3  | 2 3 3 |
| 4  | 3 1 4 |
| 5  | 3 4 5 |
| 6  | 4 5 8 |
| 7  | 5 6 2 |
| 8  | 6 4 1 |
| 9  | 3 2 4 |
| 10 | 5 4 4 |
| 11 | 3 3 6 |

3.2.4.4 输出样例 2

1 | 15

2 | 16

3 | 17

4 | -1

5 | 15

6 | 17

7 | 18

8 | 15

9 | 15

10 | 15

3.2.5 数据范围

| 子任务编号 | 测试点占比 | $n$         | $m$                  | 特殊性质    |
|-------|-------|-------------|----------------------|---------|
| 1     | 20%   | $\leq 50$   | $\leq 100$           | -       |
| 2     | 30%   | $\leq 10^5$ | $\leq 10^5$          | $n = m$ |
| 3     | 30%   | $\leq 500$  | $\leq 2 \times 10^4$ | -       |
| 4     | 20%   | $\leq 10^5$ | $\leq 10^5$          | -       |

对于所有测试点，保证  $1 \leq n \leq 10^5$ ， $1 \leq m \leq 10^5$ ， $1 \leq u_i, v_i \leq n$ ， $1 \leq w_i \leq 10^9$ 。

3.2.6 参考程序